



Wrocław University of Technology

Planning and STRIPS Representation

Halina Kwaśnicka, Maciej Piasecki
Wroclaw University of Technology
Halina.kwasnicka@pwr.edu.pl

Planning

- **Planning** - a technique of problem-solving, it is aimed at searching for a sequence of actions that causes the transition of a system from an initial state to a goal state
- **Similarity** (planning $\leftrightarrow$ inference using a set of rules):
with some representations, both techniques use matching to define a set of applicable rules or operations/actions
- **Difference:**
 - representations for planning are usually more complicated
 - planning is a noncommutative system; it requires a *revocable* control strategy
 - conflict subgoals are produced

STRIPS - Stanford Research Institute Problem Solver

- STRIPS - a language:
 - classical, often used approach
 - it allows to define efficient planning algorithms
 - can be translated to state calculus
- STRIPS - states and goals
 - **state** - conjunction of basic literals without variables (no function symbols with fixed predicates, negation is possible)
 - example: $\text{At}(\text{Home}) \wedge \text{Have}(\text{Milk})$
 - Common assumption: if a literal is not listed as a positive one in the description of a state (True), it is assumed to be False (closedness of the world)
 - **goal**: the conjunction of literals, which may contain variables
 - example: $\text{At}(x) \wedge \text{Sels}(x, \text{Milk})$
 - Common assumption: variables contain existential quantifiers

STRIP

- State
 - conjunction of atomic logical formulae
 - closed world assumption

$$\begin{aligned} & \text{At}(P1, \text{JFK}) \wedge \text{At}(P2, \text{SFO}) \\ & \wedge \text{Plane}(P1) \wedge \text{Plane}(P2) \\ & \wedge \text{Airport}(\text{JFK}) \wedge \text{Airport}(\text{SFO}) \end{aligned}$$

- Goal
 - partially described state

STRIPS representation of actions (operators)

- **Operator** (action) in STRIPS consists of:
 - **Property (description) of an action (1)**: name and meaning description
 - **Preconditions (2)**: conjunction of true literals (atoms)
 - **Effect**: conjunction of literals (true or false) describing changes of situation after using the action (operator)
 - **Effect representation**:
 - **positive: ADD list (3)** - no duplication in the state
 - **negative: DELETE list (4)**

STRIP Operators

- Variables
 - logical
 - receive value only by unification
 - introduced only in the header
- Example

```
Action(Fly(p, from, to),  
    PRECOND:At(p, from)  $\wedge$  Plane(p)  $\wedge$  Airport(from)  
 $\wedge$  Airport(to)  
    EFFECT: $\neg$ At(p, from)  $\wedge$  At(p, to))
```

STRIP Limitations

STRIPS Language	ADL (Action Description Language)
Only positive literals in states: <code>Poor</code> $\wedge$ <code>Unknown</code>	Positive and negative literals in states: $\neg$ <code>Rich</code> $\wedge$ $\neg$ <code>Famous</code>
Closed World Assumption: Unmentioned literals are false.	Open World Assumption: Unmentioned literals are unknown.
Effect <code>P</code> $\wedge$ $\neg$ <code>Q</code> means add <code>P</code> and delete <code>Q</code> .	Effect <code>P</code> $\wedge$ $\neg$ <code>Q</code> means add <code>P</code> and $\neg$ <code>Q</code> and delete $\neg$ <code>P</code> and <code>Q</code> .
Only ground literals in goals: <code>Rich</code> $\wedge$ <code>Famous</code>	Quantified variables in goals: $\forall x \text{ At}(\text{P1}, x) \wedge \text{At}(\text{P2}, x)$ is the goal of having <code>P1</code> and <code>P2</code> in the same place.
Goals are conjunctions: <code>Rich</code> $\wedge$ <code>Famous</code>	Goals allow conjunction and disjunction: $\neg$ <code>Poor</code> $\wedge$ (<code>Famous</code> $\vee$ <code>Smart</code>)
Effects are conjunctions.	Conditional effects allowed: when <code>P</code> : <code>E</code> means <code>E</code> is an effect only if <code>P</code> is satisfied.
No support for equality.	Equality predicate (<code>x=y</code>) is built in.
No support for types.	Variables can have types, as in (<code>p</code> : <code>Plane</code>).

STRIPS - plan representation

Plan - a data structure consisting of:

- **Set of plan steps** - every step is one of the possible operators
- **Set of constraints on the operators ordering**: is a specification that one of the actions must occur before another
- set of binding constraints on the variables - they are in the form $v = x$, where v is a variable in a certain step and x is a variable or constant
- **a set of causal links (connections)**: it describes the objectives of the specific steps, e.g., a goal S_i is to achieve a condition c of the state S_j

Principles of generating plans in STRIPS

- Apply the principle of the **least commitment**
 - select only what is important now - leave the rest for later
 - leave the order of steps (sequence) undefined, if it is irrelevant at the given moment
 - a plan, in which some steps are ordered, and the others are not, is a **Partial Order Plan** in contrast to a **Total Order Plan** (Linear Order Plan)
- Assign a fixed value to a variable:
example: **At(Robot,x)** **At(Robot, Home)**
- Fully concrete plans: each variable has been assigned a value

Solution in STRIPS

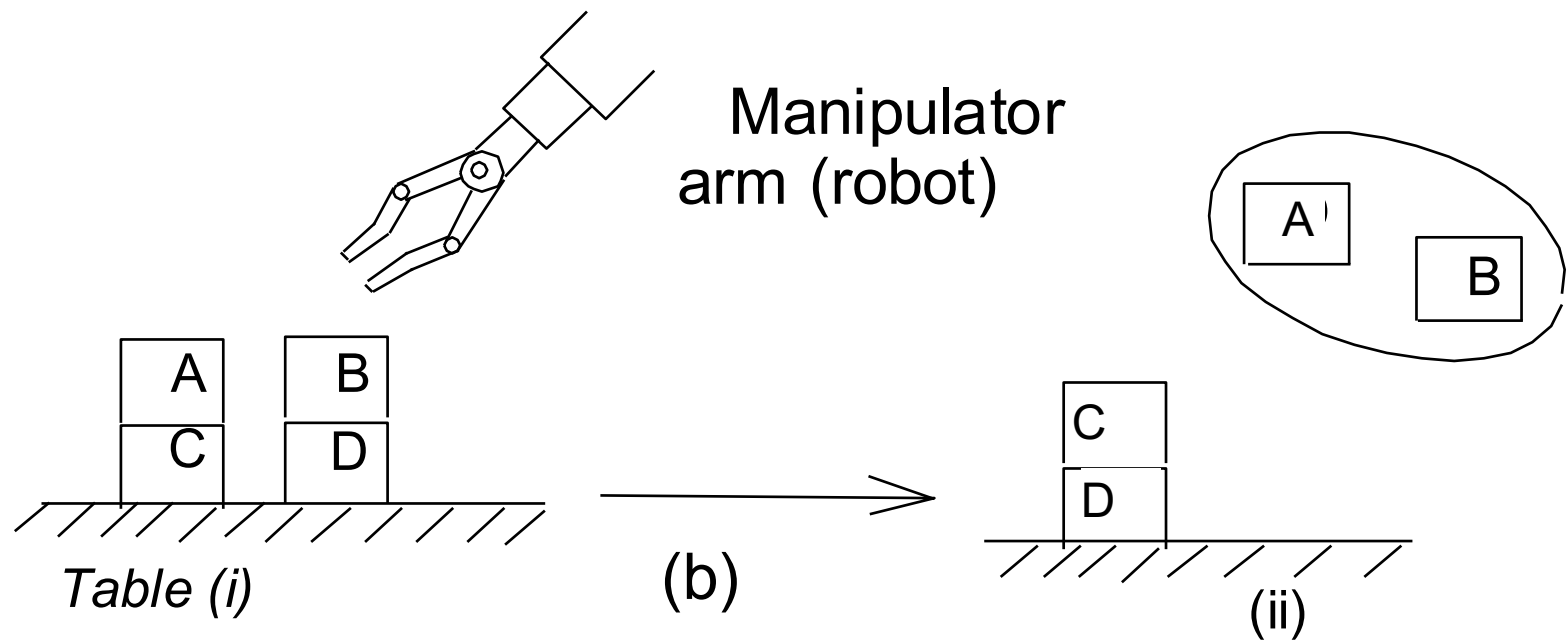
Solution: executable plan, ensuring goal achievement

- It is easy to ensure a totally ordered plan without variables
- It is not satisfactory because:
 - It is easier for 'planners' to produce a partially ordered plan than to choose an arbitrary one from the linearly ordered alternatives
 - some agents may perform actions in parallel
 - we may want to be able to manage the implementation more easily of the plan in case other restrictions occur
- Therefore, the solution is a complete and coherent plan:
 - **Complete plan:**
 - each precondition of each step is achievable by other steps
 - prerequisites are produced by other steps only when needed
 - **Consistent plan:**
 - includes no contradictions in the order of the actions and values of variables

Planning as searching

- Planning encompasses a ‘search problem’ (selection of operators), which is ubiquitous in AI
 - problem of conflict subgoals
 - to achieve a goal, a conjunction of subgoals achieved by subplans is required
 - those subplans often lead to conflicts in a desired state, e.g., (on A B) and (on B A)
 - such effects often occur when planning is parallel
- For n operators, there are $n!$ possible “orderings”
 - potentially high computational complexity

Example: World Description



Initial and goal states

Description of the states - the initial and final

Initial state S_i :

- (on A C) clause (on x y) - block x is on block y
- (on B D) predicate (clear x) - on block x is nothing
- (ontable C) predicate (ontable x) - block x is on a table
- (ontable D)
- (clear A) predicate (clear x) on block x is nothing
- (clear B)
- (empty) manipulator arm is currently empty

Goal state S_g :

- (on C D)
- (clear C)
- (ontable D)
- lack of information about the blocks A and B, nothing is required for the arm

Block-world: Operator Specification

A: Table-based block manipulation operators

1. (pickup x)

action: pickup block x (from the table)

preconditions: (ontable x), (clear x), (empty)

delete facts: (ontable x), (clear x), (empty)

add facts: (hold x)

2. (putdown x)

action: puts block x down (on the table).

preconditions: (hold x).

delete facts: (hold x).

add facts: (ontable x), (clear x), (empty).

Block-world: Operators (cont.)

B: Block-based block manipulation operators

3. **(takeoff x y)** action: takes block x off block y.
 precondition: (on x y), (clear x), (empty).
 delete facts: (on x y), (clear x), (empty).
 add facts: (clear y), (hold x).
4. **(puton x y)** action: puts block x on top of block y.
 preconditions: (hold x), (clear y).
 delete facts: (hold x), (clear y).
 add facts: (on x y), (clear x), (empty)

Partially Ordered Plan (POP): Example

- Problem: a shopping agent
 - Agent's task: to buy milk, bananas, and a drill, as well as bring them back home

- Actions:

Op (ACTION:Start,

EFFECT:At (Home) $\wedge$ Sells (HWS, Drill)

$\wedge$ Sells (SM, Milk) $\wedge$ Sells (SM, Banana))

Op (ACTION:Finish,

PRECOND:Have (Drill) $\wedge$ Have (Milk)

$\wedge$ Have (Banana) $\wedge$ At (Home))

Op (ACTION:Go (there) ,

PRECOND:At (here) ,

EFFECT:At (there) $\wedge$ $\neg$ At (here))

Op (ACTION:Buy (x) ,

PRECOND:At (store) $\wedge$ Sells (store,x) ,

EFFECT: Have (x))



POP: Searching

Start

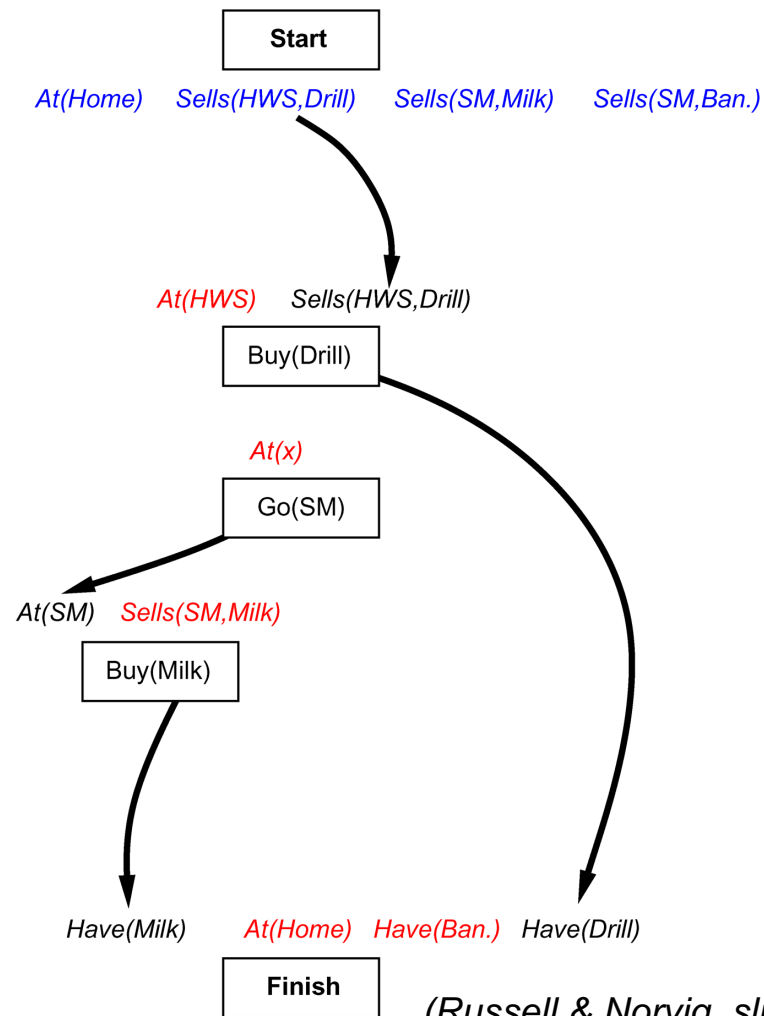
At(Home) Sells(HWS,Drill) Sells(SM,Milk) Sells(SM,Ban.)

Have(Milk) At(Home) Have(Ban.) Have(Drill)

Finish

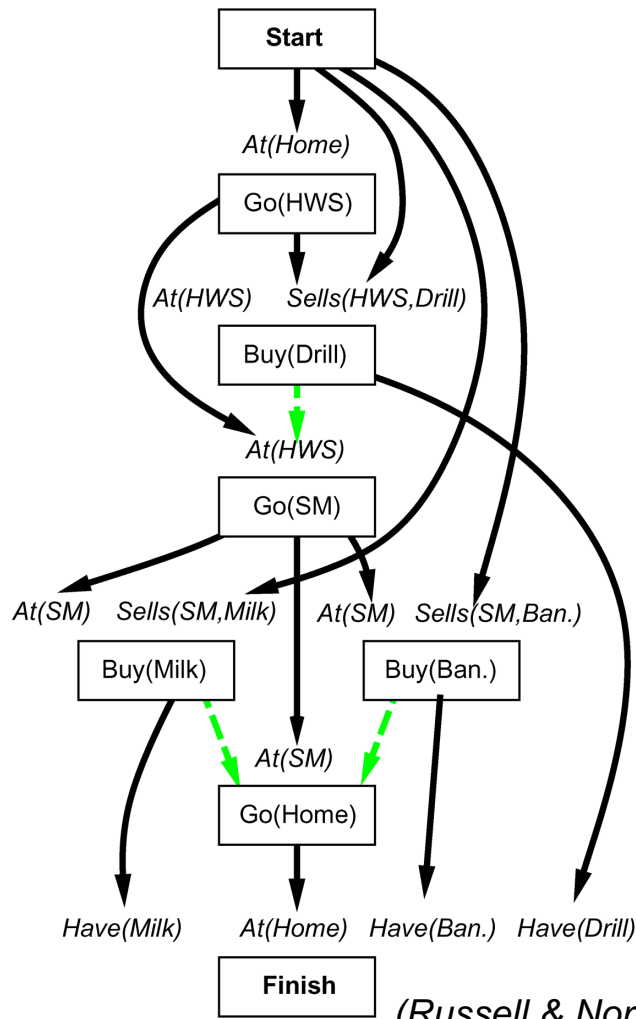
(Russell & Norvig, slides 7÷9 for Chapt. 11, 1998)

POP: Searching



(Russell & Norvig, slides 7÷9 for Chapt. 11, 1998)

POP: Searching



(Russell & Norvig, slides 7÷9 for Chapt. 11, 1998)

POP: Protection of causal links

- Threats

- a conflict realised in a partial plan by one of the actions
- actions that can **clobber** (i.e. delete) some subgoal achieved by a given link
 - e.g. $A(p) \rightarrow B$ is destroyed by C , whose effect is $\neg p$
- removing threats:
 - by adding appropriate linear order constraints

- Techniques for removing threats

- **demotion**:
 - putting C before A
- **promotion**:
 - placing C after B

POP: scheme of the algorithm without variables

function POP(*initial*, *goal*, *operators*) **returns** *plan*

plan $\leftarrow$ MAKE-MINIMAL-PLAN(*initial*, *goal*)

loop do

if SOLUTION?(*plan*) **then return** *plan*

$S_{need}, c \leftarrow$ SELECT-SUBGOAL(*plan*)

 CHOOSE-OPERATOR(*plan*, *operators*, S_{need} , *c*)

 RESOLVE-THREATS(*plan*)

end

function SELECT-SUBGOAL(*plan*) **returns** S_{need}, c

 pick a plan step S_{need} from STEPS(*plan*)

 with a precondition *c* that has not been achieved

return S_{need}, c

POP: scheme of the algorithm without variables

procedure CHOOSE-OPERATOR($plan, operators, S_{need}, c$)

choose a step S_{add} from $operators$ or $STEPS(plan)$ that has c as an effect

if there is no such step **then fail**

 add the causal link $S_{add} \xrightarrow{c} S_{need}$ to $LINKS(plan)$

 add the ordering constraint $S_{add} \prec S_{need}$ to $ORDERINGS(plan)$

if S_{add} is a newly added step from $operators$ **then**

 add S_{add} to $STEPS(plan)$

 add $Start \prec S_{add} \prec Finish$ to $ORDERINGS(plan)$

procedure RESOLVE-THREATS($plan$)

for each S_{threat} that threatens a link $S_i \xrightarrow{c} S_j$ in $LINKS(plan)$ **do**

choose either

Demotion: Add $S_{threat} \prec S_i$ to $ORDERINGS(plan)$

Promotion: Add $S_j \prec S_{threat}$ to $ORDERINGS(plan)$

POP: scheme of the full algorithm with variables

- Variables in POP
 - occur in pre-conditions and action effects
 - indirectly in states
 - behave like variables in logic
 - unification
 - value binding
 - binding removal during backtracking

Unification

- Examples of application (*Prolog notation*)

- `? X = 2.`
`X = 2`
- `? X = Y.`
`X = _G195`
`Y = _G195`
- `X = like(john).`
`X = like(john)`
- `posses(X, book(wuthering_hights, bronte)) =`
`posses(john, book(wuthering_hights, Y)).`
`X = john`
`Y = bronte`
- `X = like(john, X)`
`X = like(john, like(john, lubi(john, like(john,`
`like(john, like(john, like(john, like(john,`
`like(..., ...))))))))) ;`
- **Unification is not possible as the *occurs check* is not fulfilled**
- `a( X, c(d,X) ) = a(2, c(d,Y)).`
`X=2 Y=2.`
- `a(X, Y) = a(b(c,Y), Z).`
`X = b(c, _G304) Y = _G304 Z = _G304`

Unification

- $\text{UNIFY}(p, q) = \theta$, where:
 - $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$
 - θ is **the Most General Unifier** (MGU)
- θ is MGU for p and q if and only if:
 - $p \theta = q \theta$
 - $\forall \sigma. (p \sigma = q \sigma \rightarrow \exists \beta. (p \sigma = (q \theta) \beta))$

Unification: algorithm

function UNIFY(x, y, θ) **returns** a substitution to make x and y identical

inputs: x , a variable, constant, list, or compound

y , a variable, constant, list, or compound

θ , the substitution built up so far (optional, defaults to empty)

if $\theta = \text{failure}$ **then return** failure

else if $x = y$ **then return** θ

else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)

else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)

else if COMPOUND?(x) **and** COMPOUND?(y) **then**

return UNIFY(ARGS[x], ARGS[y], UNIFY(OP[x], OP[y], θ))

else if LIST?(x) **and** LIST?(y) **then**

return UNIFY(REST[x], REST[y], UNIFY(FIRST[x], FIRST[y], θ))

else return failure

Unification: algorithm

function UNIFY-VAR(var, x, θ) **returns** a substitution

inputs: var , a variable

x , any expression

θ , the substitution built up so far

if $\{var/val\} \in \theta$ **then return** UNIFY(val, x, θ)

else if $\{x/val\} \in \theta$ **then return** UNIFY(var, val, θ)

else if OCCUR-CHECK?(var, x) **then return** failure

else return add $\{var/x\}$ to θ

POP: scheme of the full algorithm with variables

- Possible threats

- A negative atom with an unbound variable in the effect, e.g. $\neg \text{At}(\text{here})$ vs $\text{At}(\text{Home})$
- strategies
 - immediately resolving with an equality condition
 - e.g. adding a binding $x = \text{HWS}$ to substitution
 - immediately resolving with an inequality condition
 - E.g. adding $x \neq \text{Home}$ to the substitution
 - » Necessary expansion of the conditions
 - Delayed resolving
 - Ignoring till x_{get} gets bound

POP: scheme of the full algorithm with variables

- Modification of the notion of solution
 - action A achieves a pre- p of B only if
 1. $A \prec B$, and it is necessary that the effect of A unifies with p
 2. There is no action C in the plan, such that $A \prec C \prec B$, in some linear order of the plan, and it is possible that the effect of C unifies with $\neg p$
 - Potential unbound variables in `Start`
 - Should we modify `Solution`?
 - Checking unbound variables in the plan
 - Indeterministic substitution

POP: algorithm with variables (fragments)

- Changes in the operator selection procedure:

procedure CHOOSE-OPERATOR(*plan*, *operators*, S_{need} , c)

choose a step S_{add} from *operators* or STEPS(*plan*) that has c_{add} as an effect
such that $u = \text{UNIFY}(C, c_{add}, \text{BINDINGS}(\textit{plan}))$

if there is no such step

then fail

add u to BINDINGS(*plan*)

add $S_{add} \xrightarrow{c} S_{need}$ to LINKS(*plan*)

add $S_{add} \prec S_{need}$ to ORDERINGS(*plan*)

if S_{add} is a newly added step from *operators* **then**

add S_{add} to STEPS(*plan*)

add $\text{Start} \prec S_{add} \prec \text{Finish}$ to ORDERINGS(*plan*)

(Russell & Norvig, chapt. 11, pp. 358, 1995)

POP: algorithm with variables (fragments)

- Changes in the procedure for resolving conflicts:

```
procedure RESOLVE-THREATS(plan)  
  for each  $S_i \xrightarrow{c} S_j$  in LINKS(plan) do  
    for each  $S_{threat}$  in STEPS(plan) do  
      for each  $c'$  in EFFECT( $S_{threat}$ ) do  
        if SUBST(BINDINGS(plan),  $c$ ) = SUBST(BINDINGS(plan),  $\neg c'$ ) then  
          choose either  
            Promotion: Add  $S_{threat} \prec S_i$  to ORDERINGS(plan)  
            Demotion: Add  $S_j \prec S_{threat}$  to ORDERINGS(plan)  
          if not CONSISTENT(plan)  
            then fail  
          end  
        end  
      end  
    end  
  end
```

Subplans - Triangular Table

- To remember certain steps in the plan, which may be common for different purposes (S_{g1} - to work, S_{g2} - club, S_i - home)
- Each of the plans for S_{g1} and S_{g2} has a common sequence of operators (to go to the car, start the engine, etc.)
- Triangle Table* - a mechanism for cataloguing plans and operators' interaction

0	Initial state. S_i	1: O_1	
1	The contents of the cell above minus <i>delete facts</i> of operator O_1	<i>Add facts of O_1</i>	2: O_2
2	The contents of the cell above minus <i>delete facts</i> of operator O_2	The contents of the cell above minus <i>delete facts</i> of operator O_2	<i>Add facts of O_2</i>
			3: O_3
			P: O_p
p			<i>Add facts of O_p</i>

Triangular Table Size

	0	1	2	3	4	5
0						
1						
2						
3						
4						
5						

- For plans consisting of p operators $O_1, O_2, \dots, O_p$, table consists of:
- $p+1$ column numbered from 0 to p
- $p+1$ rows, from 0 to p
- Number of cells in the table is :

$$k = \frac{(p+1)^2}{2} + \frac{p+1}{2} = p \cdot \frac{p+3}{2} + 1$$

- For $p=4$ it is $4 \cdot 7 / 2 + 1 = 15$ cells, $p=5$: 21 cells

- A cell can be empty or contain a subset of clauses describing the state of the system
- The cell $(0,0)$ contains a description of the initial state S_i
- The last row contains a description of the target S_g

T Table for a Plan in a Blocks World

0	on BD, ontable C ontable D, clear A clear B, empty	1	takeof AC							
1	clear B, on BD ontable C, ontable D	hold A clear C	2	putdown A						
2	on BD, clear B, ontable C,ontable D	clear C	clear A, empty ontable A	3	pickup C					
3	on BD, clear B, ontable D	_____	clear A ontable A	hold C	4	puton CA				
4	on BD, clear B ontable D	_____	ontable A	_____	clear C, empty, on CA	5	takeoff BD			
5	ontable D	_____	ontable A	_____	clear C on CA	clear D hold B	6	putdown B		
6	ontable D	_____	ontable A	_____	clear C on CA	clear D	empty, clear B ontable B	7	takeoff CA	
7	ontable D	_____	ontable A	_____	_____	clear D	clear B ontable B	hold C clear A	8	puton CD
8	ontable D	_____	ontable A	_____	_____	_____	clear B ontable B	clear A	empty on CD clear C	

Planning Domain Definition Language

- Two files for a planning task:
 - a domain file for predicates and actions.
 - a problem file for objects, initial state and goal specification.

- Domain files

```
(define (domain <domain name>)  
  (:requirements :strips)  
  (:predicates  
    <list of predicate schemata> )  
  <list of action schemata>  
)
```

PDDL - Problem Files

- Scheme

```
(define (problem <problem name>)
  (:domain <domain name>)
  (:objects
    <list of objects>
  )
  (:init
    <predicates that hold in initial state>
  )
  (:goal
    <goal description>
  )
)
```

PDDL - Example - Textual description

„There is a robot that can move between two rooms and pick up or drop balls with either of its two arms. Initially, all balls and the robot are in the first room. We want the balls to be in the second room.”

- **Objects:** The two rooms, four balls and two robot arms.
- **Predicates:** Is x a room? Is x a ball? Is ball x inside room y ? Is robot arm x empty? [...]
- **Initial state:** All balls and the robot are in the first room. All robot arms are empty. [...]
- **Goal specification:** All balls must be in the second room.
- **Actions/Operators:** The robot can move between rooms and pick up or drop a ball.

PDDL - Example

- Objects

```
(:objects rooma roomb  
ball1 ball2 ball3 ball4  
left right)  
)
```

- Predicates

```
(:predicates (ROOM ?x) (BALL ?x) (GRIPPER ?x)  
(at-robbby ?x) (at-ball ?x ?y)  
(free ?x) (carry ?x ?y)  
)
```

PDDL - Example

- Initial state

```
(:init (ROOM rooma) (ROOM roomb)
(BALL ball1) (BALL ball2) (BALL ball3) (BALL ball4)
(GRIPPER left) (GRIPPER right) (free left) (free
right)
(at-robby rooma)
(at-ball ball1 rooma) (at-ball ball2 rooma)
(at-ball ball3 rooma) (at-ball ball4 rooma))
)
```

PDDL - Example

- Goal

```
(:goal (and (at-ball ball1 roomb)  
(at-ball ball2 roomb)  
(at-ball ball3 roomb)  
(at-ball ball4 roomb))
```


PDDL - Example - Operators

- Movement operator

```
(:action move :parameters (?x ?y)
  :precondition (and (ROOM ?x) (ROOM ?y)
    (at-robby ?x))
  :effect (and (at-robby ?y) (not (at-robby ?x))))
)
```

- Pick-up operator

```
(:action move :parameters (?x ?y)
  :precondition (and (ROOM ?x) (ROOM ?y)
    (at-robby ?x))
  :effect (and (at-robby ?y) (not (at-robby ?x))))
)
```

PDDL - Example - Operators

- Drop operator

```
(:action drop :parameters (?x ?y ?z)
  :precondition ( and (BALL ?x) (ROOM ?y)
    (GRIPPER ?z) (carry ?z ?x) (at-robby ?y) )
  :effect (and (at-ball ?x ?y) (free ?z)
    (not (carry ?z ?x)))
)
```

PDDL - Versions

- 1998: PDDL 1.2 (basic version)
- 2002: PDDL 2.1 (numeric and temporal features)
- 2004: PDDL 2.2 (derived predicates and timed initial literals)
- 2004: PPDDL (probabilistic)
- 2006: PDDL 3 (soft goals and trajectory constraints)
- 2006: PDDL+ (continuous state spaces)

PDDL - Selected extensions

- Not all extensions are supported by different types of planning algorithms
- Types for objects and arguments
- Universally quantified effects
`(forall (?v1 ... ?vn) <effect> )`
- Conditional effects
`(when <condition> <effect>))`

PDDL - Selected extensions

- Action cost

```
(define (domain <domain name>)
```

```
  (:requirements :strips :action-costs)
```

- plus: initial cost, effect cost

```
  (increase (total-cost) <action cost>)
```

- functions, numerical conditions, numerical operations on cost values

Planning as State-Space Search

- Satisficing or Optimal Planning
 - **satisficing planning**: any plan is OK (cheaper ones preferred)
 - **optimal planning**: plans must have a minimum cost
 - almost no overlap in the algorithms
- Main techniques
 - **explicit search** - only heuristics are practical
 - progression: forward from the initial state to the goal
 - regression: backward from goal states to the initial state
 - SAT planning
 - symbolic search

Explicit search

- Search across a set of states
 - states represented by propositions (complex logical sentences)
 - propositional logic
- Only heuristics are practical
 - progression: forward from the initial state to the goal
 - regression: backward from goal states to the initial state

SAT Planning

- SAT - algorithms that find satisfying assignments to CNF (Conjunctive Normal Form) formulas
- Formalisation of a plan as a propositional satisfiability problem
 - propositional variables for states - v_i -th state
 - propositional variables for operators - o_i -th operator in a sequence
 - to express a formula whose models are exactly the plans
- Design choices:
 - SAT encoding, SAT solver, evaluation strategy
- Examples: MiniSAT, Glucose, Lingeling

Symbolic Search

- Search processes sets of states at a time
 - sequences of operators
 - across state sets reachable with a given cost.
 - search space represented by Symbolic Data Structures
 - e.g. binary decision diagrams (BDDs)
- Decisions
 - search direction, algorithm, control (and heuristics)

Heuristics for Planning by Searching

- Delete relaxation
 - ignoring all delete effects from operators
 - relaxed planning for calculating ‘distance’
- Abstraction
 - dropping some distinctions between states, while preserving the transition behaviour as much as possible
- Constraint-based Heuristics
 - Landmarks
 - exploration of constraints that hold in every solution of the task
 - Network flows
 - properties of a sequence of actions, e.g. balance: consumption vs production
 - Potential heuristics
 - based on a linear combination of the features describing states

Planning systems: FF

- problem class: satisficing
- algorithm class: explicit search
- search direction: forward search
- search algorithm: enforced hill-climbing
- heuristic: FF heuristic (inadmissible)
- other aspects: helpful action pruning; goal agenda manager

Planning systems: LAMA

- problem class: satisficing
- algorithm class: explicit search
- search direction: forward search
- search algorithm: restarting Weighted A* (anytime)
- heuristic: FF heuristic and landmark heuristic (inadmissible)
- other aspects: preferred operators; deferred heuristic evaluation; multi-queue search

Planning systems: Fast Downward

Stone Soup

- problem class: optimal
- algorithm class: (portfolio of) explicit search
- search direction: forward search
- search algorithm: A^*
- heuristic: LM-cut; merge-and-shrink; landmarks;
- blind (admissible)

Further Readings

- <https://planning.wiki/>
- PLANNING ALGORITHMS, Steven M. LaValle
University of Illinois Copyright Steven M. LaValle
2006, <http://planning.cs.uiuc.edu/> Published by
Cambridge University Press
- Jendrik Seipp. TDDD48 Automated Planning -
Teaching Machines to Think, Access 28th March
2025: <https://mrlab.ai/tddd48/>